
kidbox

An Architectural Field Guide

How a locked-down, retro kid-computer is built
out of a stack of small, explicit Unix parts

A deep but companionable tour of the boot chain, the console, the X Window System, systemd, the input subsystem, the power button, and the typesetting of the instruction booklet — with frequent detours into how these technologies came to be, and into roads the project chose not to take. Written to be printed, folded under one arm, and read in bed.

Contents

1	How to read this report	2
2	The shape of the whole system	2
3	From power-on to a menu: the boot and login chain	3
3.1	Firmware, kernel, and the hand-off to <code>init</code>	3
3.2	<code>getty</code> , <code>agetty</code> , and automatic login	4
3.3	Login shells, and why the menu lives in <code>.bash_profile</code>	5
3.4	Making the console legible: console fonts and blanking	6
4	The menu: <code>whiptail</code>, <code>newt</code>, and a button that hides	6
4.1	Drawing a menu with <code>whiptail</code>	6
4.2	Hiding the “OK” button: the <code>NEWT_COLORS</code> trick	7
4.3	Refusing to die: signal traps and the missing <code>exit</code>	7
5	Launching graphics on demand: the X Window System	8
5.1	The <code>run_x</code> helper and <code>xinit</code>	8
5.2	Inside <code>.xinitrc</code> : a desktop in eleven lines	9
5.3	Big fonts everywhere: <code>.Xresources</code> , <code>Xft</code> , and DPI	10
5.4	The one escape hatch: <code>Ctrl+Alt+Backspace</code> via <code>xbindkeys</code>	10
6	The activities	11
6.1	Two ordinary desktop programs	11
6.2	Three vintage programming environments	11
6.3	Three home-grown activities	12
6.4	Sound: <code>ALSA</code> , <code>amixer</code> , and <code>mpg123</code>	13
7	The power button: a small system unto itself	13
7.1	The problem and the three-part solution	13
7.2	Part 1: telling <code>logind</code> to stand down	14
7.3	Part 2: watching the button with <code>evdev</code>	14
7.4	Part 3: the confirmation menu on its own VT	15
7.5	The service unit, and what its hardening buys	16
8	Permissions: a deliberately small amount of power	17
9	Installation: one idempotent shell script	17
9.1	Safety rails at the top	17
9.2	Idempotency: safe to run again and again	18
9.3	The marker-block trick for <code>.bash_profile</code>	18
9.4	Then it wires up the system	18
10	The instruction booklet: <code>LaTeX</code> and a little bookbinding	19
10.1	A book sized for small hands	19
10.2	From PDF to bootable booklet: imposition	19
11	The development story in the repository	20
	Appendix A: File map	20
	Appendix B: Glossary	20

1 How to read this report

This document explains, in more detail than is strictly necessary, how the **kidbox** project turns a Raspberry Pi into a deliberately simple computer for two children. You asked for something detailed yet interesting: a piece of professional development you can enjoy rather than endure. So alongside the literal description of what each file does, there are three kinds of marginal excursion you will see throughout:

Going deeper

A **Going deeper** box steps back from kidbox itself to explain the technology underneath — where it came from, why it works the way it does, and what else it can do. These are the parts that are fun to know even though the project would run fine if you never read them.

Try this

A **Try this** box suggests a harmless experiment you can run on the Pi (or any Linux box) to see a concept with your own eyes. None of them are required reading.

Field note

A **Field note** flags something I noticed while reading the code: a latent bug, a rough edge, a place where the comments and the behaviour have drifted apart. You said you vibe-coded most of this and fix the “AI slop” yourself, so these are offered in that spirit — small observations, not criticisms.

The big idea of kidbox, stated in its own README, is worth keeping in mind the whole way through:

“Intentionally simple and explicit. The idea is that the only way to use such a computer is to start understanding how it works.”

That philosophy is not just a slogan about the *child’s* experience. It also describes the *engineering*: kidbox is assembled from many small, old, well-understood Unix tools, each doing one visible thing, wired together with shell scripts you can read in an afternoon. There is no framework, no container, no daemon you can’t cat. That makes it an unusually good specimen for a guided dissection.

2 The shape of the whole system

Before the details, here is the system in one breath. A Raspberry Pi powers on, boots Linux, and *automatically logs in* a passwordless user named `girls` on the first virtual terminal. That user’s login shell runs a snippet that enlarges the console font and launches a text menu drawn by `whiptail`. The menu is the entire user interface. When a child picks an activity, the menu starts a *throwaway* graphical X session containing exactly one program — Tux Paint, say, or the Logo turtle — and nothing else: no desktop, no taskbar, no way out except finishing. When the program quits, X is torn down and the child is back at the menu. A background service watches the physical power button so that a careless poke shows a confirmation dialog instead of yanking the power.

Everything kidbox adds to a stock Raspberry Pi OS falls into a few buckets:

- **A user and its login wiring** — the `girls` account, console autologin, a font, and a menu launched from `.bash_profile`.
- **A menu** — one bash script (`menu.sh`) using `whiptail` for the list and a helper to launch X programs.
- **On-demand X sessions** — `xinit` plus a tiny `.xinitrc` that starts a minimal window manager and one application.
- **A handful of activities** — two are ordinary desktop programs (Tux Paint, Leafpad), three are vintage

programming environments (UCBLogo, PC-BASIC), and three are home-grown (an HTML clock, a shell timer, a shell stopwatch).

- **Power-button safety** — a `systemd-logind` tweak plus a small Python service reading the kernel’s input devices.
- **An instruction booklet** — a LaTeX document typeset and imposed for saddle-stitch printing.

Figure 1 shows how the pieces relate.

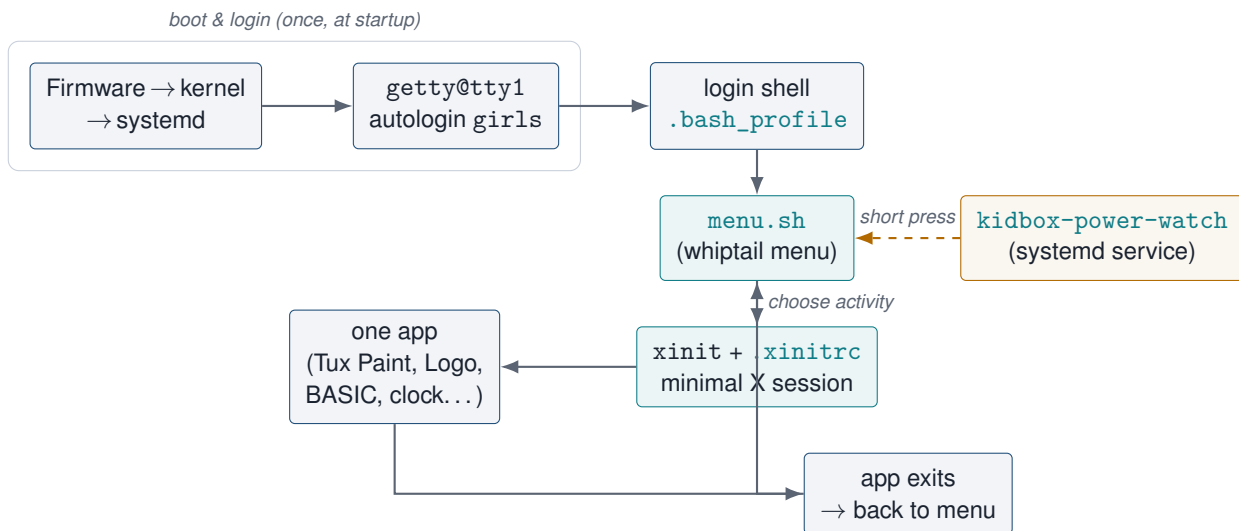


Figure 1: The life of the machine. Boot and login happen once; the menu → app → menu loop runs forever. The power watcher sits to the side, waking only when the button is pressed.

Going deeper: why “one program, then throw it away”?

Most computers run a *session*: a desktop that persists, with many windows and a way to switch between them. kidbox deliberately refuses this. Each activity gets a fresh, single-window X server that exists only for the lifetime of that one program. This is the same trick a *kiosk* uses — an airport check-in screen, a museum exhibit — and it buys three things at once: *isolation* (a crash takes down only that activity), *simplicity* (there is no window management to learn or escape), and *statelessness* (every activity starts clean). The cost is start-up latency: spinning up an X server takes a second or two. For a child choosing one thing at a time, that trade is clearly worth it.

3 From power-on to a menu: the boot and login chain

This is the most “Linux internals” part of the project, and the part you said you’d not had to configure before. Let’s walk the whole chain.

3.1 Firmware, kernel, and the hand-off to `init`

When a Raspberry Pi powers on, its *firmware* (on the Pi, code running on the VideoCore GPU, not a traditional PC BIOS/UEFI) reads the boot partition, loads the Linux kernel, and jumps into it. The kernel brings up hardware, mounts the root filesystem, and then starts exactly one user-space process, historically called `init`, with process ID 1. On Raspberry Pi OS (a Debian derivative) that process is **systemd**.

`systemd`’s job is to bring the system up to a *target* — a named collection of services that should be running. kidbox cares about `multi-user.target` (a fully booted, networked, text-mode system) rather

than `graphical.target` (which would start a graphical login manager). kidbox never installs a graphical login manager, so the machine naturally rests at the multi-user, text-console state. That is the foundation the whole design rests on: **boot to a console, not to a desktop**.

Going deeper: targets, and the ghost of runlevels

If you came up on older Unix or Linux, you remember *runlevels*: a single number (0–6) describing how much of the system was running, set in `/etc/inittab`. Runlevel 3 was “multi-user text mode,” runlevel 5 added graphics, runlevel 0 was halt, 6 was reboot. `systemd` replaced this with *targets*, which are just units that depend on other units, so the model is a graph rather than a ladder. The old names survive as compatibility aliases: `runlevel3.target` is `multi-user.target`. You can ask the running system what it’s aiming for with `systemctl get-default`, and where it is right now with `systemctl list-units --type target`.

3.2 getty, agetty, and automatic login

A text console is offered to you by a program called a **getty**. The name is a fossil: it stands for “get tty,” and a *tty* is in turn short for *teletypewriter* — the electromechanical printing terminals that were the first interactive Unix consoles in the early 1970s. The getty’s classic job was to watch a terminal line, print the login: prompt, read a username, and hand off to the `login` program to check the password.

On a `systemd` machine each text console is managed by a templated service, `getty@.service`, instantiated per virtual terminal: `getty@tty1`, `getty@tty2`, and so on. The actual program run is `agetty` (“alternative getty”). kidbox needs the first console to skip the username/password dance and log the `girls` user straight in. The README does this with a `systemd` *drop-in* override:

```
sudo systemctl edit getty@tty1
```

```
[Service]
ExecStart=
ExecStart=-/sbin/agetty --autologin girls --noclear %I $TERM
```

Two subtleties hide in those four lines, and they’re worth understanding because they trip up nearly everyone the first time:

- The **empty** `ExecStart=`. A `systemd` service may normally have only one `ExecStart`. The packaged `getty@.service` already defines one. Assigning an empty value first *clears* the inherited command, and the second line sets the replacement. Without the blank line you’d get an error about multiple `ExecStart` directives.
- The **leading dash** in `-/sbin/agetty`. In a `systemd` unit a dash before the executable means “a non-zero exit from this process is not a failure.” Gettys come and go constantly as people log in and out; the dash keeps `systemd` from treating a normal exit as a fault.

`-autologin girls` tells `agetty` to bypass authentication and run `login` for that user directly. `-noclear` stops it from clearing the screen first. `%I` is the instance name (here `tty1`), and `$TERM` carries the terminal type.

Field note

The README adds a wise warning: don’t use `raspi-config`’s `autologin` option, because that wires `autologin` to *your* user (the `pi`-style admin account), not to `girls`. Doing this by hand is the correct call. The only thing missing is a reminder that `systemctl edit` writes to

`/etc/systemd/system/getty@tty1.service.d/override.conf` and that you can inspect the merged result with `systemctl cat getty@tty1` — handy when it mysteriously doesn't take effect.

Going deeper: the virtual terminals behind Ctrl+Alt+F1

The Linux kernel multiplexes one physical screen and keyboard into a dozen independent text consoles, the *virtual terminals* (VTs). You switch between them with Ctrl+Alt+F1 through F6 (and beyond). Each is a fully separate login session. kidbox leans on this hard: the child's menu lives on tty1, and — as we'll see — the power-button confirmation dialog deliberately pops up on tty12, an otherwise-unused VT, so the two never fight over the keyboard. The current VT is readable at any time from `/sys/class/tty/tty0/active`, and you can switch programmatically with the `chvt` command. This is one of those Linux facilities that feels archaic until you need exactly it, at which point it is perfect.

3.3 Login shells, and why the menu lives in `.bash_profile`

Once login succeeds, it starts the user's *login shell* — `bash`. Here we meet a distinction that confuses almost everyone:

- A **login shell** (what you get from console login, autologin, or `ssh`) reads `/etc/profile` and then the first it finds of `~/.bash_profile`, `~/.bash_login`, or `~/.profile`.
- An **interactive non-login shell** (a new terminal window inside an already-running desktop) reads `~/.bashrc` instead.

kidbox wants its menu to appear *exactly once, at the moment of login*, so the autostart logic belongs in `.bash_profile` — and that is precisely where the installer puts it. The snippet (`config/bash_profile_snippet.sh`) is short and careful:

```
# Auto-run kid menu only on local interactive TTY (not SSH)
if [[ -t 0 && -z "${SSH_CONNECTION:-}" ]]; then
  if command -v setfont >/dev/null 2>&1; then
    (setfont ter-v32n || setfont ter-132n \
      || setfont /usr/share/consolefonts/Lat15-Terminus32x16.psf.gz) \
      >/dev/null 2>&1 || true
  fi
  setterm --blank 10 2>/dev/null || true
  "$HOME/bin/menu.sh"
fi
```

The guard `[[ -t 0 && -z "${SSH_CONNECTION:-}" ]]` is the clever bit. `-t 0` asks “is standard input a terminal?” (true on a real console, false if a script piped input in). `-z "${SSH_CONNECTION:-}"` asks “is the `SSH_CONNECTION` variable empty?” — it is set automatically by `sshd` for remote logins. Together they mean: *only launch the kiosk menu when someone is physically at the machine*. If you SSH in to fix something, you get a normal shell, not the locked-down menu. That single line is the maintenance escape hatch for the whole appliance.

Try this

On any Linux box, run `echo $SSH_CONNECTION` in an SSH session and then again on the physical console. Remote, it prints four numbers (client IP, client port, server IP, server port); local, it prints an empty line. That's the exact signal kidbox uses to tell “here” from “there.”

3.4 Making the console legible: console fonts and blanking

Two more things happen before the menu appears, both aimed at small children reading a screen from across a desk.

The font. `setfont ter-v32n` loads a Terminus console font that is 32 pixels tall — roughly double the default. The cascade of fallbacks (`ter-132n`, then an explicit path) is defensive: different Debian releases name and ship these fonts slightly differently, so the snippet tries three spellings and shrugs off failure with `|| true`. `setfont` comes from the `kbd` package, which the installer dutifully lists.

The blanking. `setterm -blank 10` tells the kernel console to go dark after 10 minutes of inactivity — a crude screensaver for the text mode, distinct from the X-level blanking we’ll meet later.

Going deeper: what a console font actually is

A graphical desktop renders text from scalable outline fonts (TrueType, OpenType) at any size. The Linux *text* console predates all that: it draws characters from a fixed bitmap *glyph table* called a PSF file (“PC Screen Font”). A PSF is almost comically simple — a tiny header, then one bitmap per glyph, each glyph a fixed grid of pixels (Terminus 32×16 means each character is 16 px wide and 32 px tall) — plus an optional Unicode mapping table that says which code points each glyph represents. Because the glyphs are fixed-size bitmaps, “making the font bigger” on the console doesn’t mean scaling; it means *loading a different file with taller glyphs*, which is exactly what `setfont ter-v32n` does. The `console-setup` package (also installed by `kidbox`) is the mechanism for making such a choice *persist* across reboots, by regenerating the font at boot; the `.bash_profile` approach instead sets it freshly at each login, which is simpler and good enough here.

4 The menu: whiptail, newt, and a button that hides

The menu is the heart of `kidbox` and a tidy little study in old-school terminal UI. It lives in `bin/menu.sh`. Structurally it is an infinite `while true` loop: draw a menu, read the choice, launch the chosen activity, and when that returns, loop back and draw the menu again. The only way out is the “Shutdown” item.

4.1 Drawing a menu with whiptail

`whiptail` draws text-mode dialog boxes — menus, message boxes, input prompts, progress bars — using box-drawing characters and colour. `kidbox` uses its `-menu` mode:

```
CHOICE=$(
  whiptail --title "$MENU_TITLE" --nocancel \
    --menu "Choose something to do" "$TERM_LINES" "$TERM_COLS" "$MENU_ROWS" \
      "${MENU_ITEMS[@]}" \
    3>&1 1>&2 2>&3
)
```

Two pieces of arcana are worth unpacking.

The 3>&1 1>&2 2>&3 dance. This is the single most head-scratching idiom in terminal scripting. `whiptail` draws its *interface* on standard output but prints the user’s *selection* on standard error — precisely so the drawing and the result don’t get mixed in a pipe. But `$(...)` captures standard output, not standard error. The juggling swaps the two streams: open a temporary file descriptor 3 pointing at the original stdout (the screen), send stdout to stderr, send stderr to fd 3. Net effect: the menu still draws on the real screen, and the *answer* flows back through `$(...)` into `CHOICE`. It looks like line noise; it is actually a precise plumbing diagram.

Sizing to the terminal. The menu reads `LINES` and `COLUMNS` (falling back to `tput lines/tput cols`) so the dialog fills the whole screen rather than floating in a small box. `MENU_ROWS` is the terminal height minus a few lines of chrome.

4.2 Hiding the “OK” button: the `NEWT_COLORS` trick

Here is my favourite hack in the whole repository, and the comments in the code already explain the constraint honestly. `whiptail` can suppress its *Cancel* button with `-nocancel`, but it offers *no* switch to hide the *OK* button. For a child-proof kiosk you don’t want a focusable “OK” to tab to and wonder about. So the author makes the button *invisible by colouring it to match the background*:

```
export NEWT_COLORS='
root=white,blue
window=white,blue
button=white,blue
actbutton=white,blue
...'
,
```

Every element is white-on-blue, including both the resting and active button colours, so the OK button is still *there* and still works — it has simply been painted into the wallpaper. This is a wonderfully pragmatic piece of “simple and explicit” engineering: rather than patch `whiptail` or switch toolkits, paint the unwanted control the colour of the wall.

Going deeper: `whiptail` vs `dialog`, `newt` vs `ncurses`

There are two great families of text-mode dialog tools, and they descend from two different terminal libraries.

`whiptail` is built on **`newt`**, a library written at Red Hat in the 1990s; the name is a half-joke, “Not Erik’s Windowing Toolkit,” after its author’s earlier “Erik’s Windowing Toolkit.” `newt` is small, opinionated, and drives its colours through the `NEWT_COLORS` environment variable `kidbox` exploits above.

The older, more featureful cousin is `dialog`, built on **`ncurses`** — the free reimplement of AT&T’s `curses` library, which itself grew out of the `vi` editor’s need to move a cursor around any terminal without caring what model it was. The magic underneath all of this is **`terminfo`**: a database (`/usr/share/terminfo`) describing, for every terminal type ever made, the exact byte sequences to move the cursor, clear the screen, or turn text blue. When you set `TERM=linux` — as the power-button script explicitly does, because a `systemd` service has no inherited terminal type — you are choosing which page of that database `curses/newt` should read. `kidbox` could have used `dialog` instead of `whiptail` and the menu would look almost identical; `whiptail` wins here purely on being tiny and already present on Raspberry Pi OS.

4.3 Refusing to die: signal traps and the missing exit

A kiosk must not be escapable by the usual “break out of a program” reflexes. `kidbox` blocks the obvious one — `Ctrl+C` — with a single line near the top:

```
if [[ "$DEV_MODE" == false ]]; then
  trap '' INT
fi
```

`trap '' INT` installs an *empty* handler for `SIGINT`, the signal `Ctrl+C` sends. An empty handler means “catch it and do nothing,” so `Ctrl+C` at the menu is simply inert. The `-dev` flag leaves the trap off (and adds an

Exit option and lets Cancel quit) so you can iterate on a normal laptop without trapping yourself in your own kiosk.

Field note

A subtle, real consequence worth knowing: `trap " INT` is *inherited* by child processes, so any program the menu launches *in the same process context* also ignores Ctrl+C. That’s fine for the menu loop, but it’s the reason the timer/stopwatch instructions say “Press Ctrl+C to stop” yet run inside their *own* `xterm`, which gets a fresh signal disposition — so there the Ctrl+C does work. The two behaviours are consistent once you trace the process tree, but it’s the kind of thing that looks contradictory until you do.

There is also no “Exit” item in production mode, and pressing Esc (which `whiptail` reports as a non-zero exit) hits the `continue` branch that just redraws the menu. Between the trap, the missing exit, and the Esc handling, there is no path from the menu back to a raw shell — exactly the intent.

5 Launching graphics on demand: the X Window System

When the child picks Tux Paint or the Logo turtle, kidbox needs pixels, not text. It gets them by starting a fresh, minimal X session around a single program. This section is the densest in the report, because the X Window System is a deep, 40-year-old piece of software and kidbox uses a surprising amount of it deliberately.

5.1 The `run_x` helper and `xinit`

The menu’s launcher is a small function:

```
run_x() {
  export KID_APP="$1"; shift || true
  export KID_ARGS="${*:~}"
  xinit -- :1 -br -nolisten tcp "vt${XDG_VTNR:-1}" >> "$LOGFILE" 2>&1
}
```

It records the chosen program in two environment variables, then calls `xinit`. `xinit` is the bare-metal way to start X: it launches the X server and one *client* (here, whatever `.xinitrc` runs), and crucially, **when that client exits, `xinit` shuts the server down**. That is the entire “throwaway session” mechanism — no display manager, no session daemon, just “start X, run one thing, tear it all down.”

The arguments after `-` go to the X server:

- `:1` — the *display number*. X servers are addressed as `:0`, `:1`, etc. kidbox uses `:1` to stay clear of any `:0`.
- `-br` — start with a black root window (no grey stipple flash).
- `-nolisten tcp` — do *not* open a network port; accept only local connections over a Unix socket. (More on why this matters below.)
- `vt$XDG_VTNR` — bind the server to the *current* virtual terminal, so graphics appear on the same screen the child is already looking at. `XDG_VTNR` is the VT number `systemd` exports for the session.

Going deeper: what “X” actually is, and why it’s a network protocol

The X Window System (X11, 1987, out of MIT’s Project Athena) is built on an idea that still surprises people: it is a **client-server protocol**, and — counter-intuitively — the *server* is the thing in front of you. The X server owns the screen, keyboard, and mouse. Programs that want to draw windows

are *clients*; they connect to the server and send requests like “draw a line” or “tell me when a key is pressed.” Because that conversation is a network protocol, the client and server need not be on the same computer — this is how, decades before “cloud,” a Unix workstation could run a program on a distant mainframe and display its windows locally. kidbox never wants that, which is exactly why it passes `-nolisten tcp`: there is no reason to accept connections from the network, so it refuses to even open the port, shrinking the attack surface to zero. The clients on this machine reach the server through a local Unix-domain socket instead. So a humble offline kid-computer quietly demonstrates the most famous (and most debated) design decision in the history of graphical Unix.

5.2 Inside `.xinitrc`: a desktop in eleven lines

When `xinit` starts the server, it runs the user’s `.xinitrc` as the session. kidbox’s is about as small as a usable graphical environment gets:

```
xset s on; xset s 600; xset +dpms; xset dpms 0 0 600    # blank after 10 min
[ -f ~/.Xresources ] && xrdp -merge ~/.Xresources        # load fonts/DPI
unclutter -idle 3 &                                    # hide idle cursor
matchbox-window-manager -use_titlebar no \
-use_lowlight -use_fullscreen &                        # minimal WM
xbindkeys &                                             # global hotkeys
if [ -n "${KID_APP:-}" ]; then
    exec "$KID_APP" ${KID_ARGS:-}                      # become the app
fi
exec xterm                                             # fallback
```

Read it top to bottom and you have a complete, if spartan, desktop:

- **Screen blanking (`xset`).** `xset s 600` sets the screensaver timeout to 600 seconds; `+dpms` enables DPMS (Display Power Management Signaling), the standard by which the graphics card tells the monitor to power down; `dpms 0 0 600` sets the standby/suspend/off timers so the display actually sleeps after ten minutes. This is the X-session twin of the console `setterm -blank` we saw earlier.
- **Resources (`xrdb`).** `xrdb -merge ~/.Xresources` loads the X resource database — a key/value store of application preferences (fonts, colours, DPI). More on this just below.
- **Cursor hiding (`unclutter`).** Hides the mouse pointer after 3 idle seconds, so a forgotten cursor doesn’t sit on the drawing.
- **Window manager (`matchbox`).** A window manager decides where windows go and draws their frames. `matchbox-window-manager` is a minimalist WM designed for small/embedded screens; kidbox starts it with *no* title bars and *fullscreen*, so the single app fills the display with no decorations to poke at.
- **Hotkeys (`xbindkeys`).** A daemon that binds key combinations to commands, independent of which app has focus (see below).
- **`exec` the app.** The script *becomes* the chosen program via `exec` (replacing the shell process). Because this is the last, foreground client, *its* exit is what tells `xinit` to bring the whole server down — closing the throwaway-session loop.

Going deeper: the window manager is not the GUI

Newcomers often assume “the window manager” *is* the desktop. It isn’t. Under X, drawing windows is the application’s job; the window manager only decides *where* top-level windows sit, draws their borders/title bars, and handles moving, resizing, and focus. That separation is why you can swap `matchbox` for `i3`, `openbox`, or a full GNOME shell without changing a single application. kidbox

exploits the separation to the limit: by choosing the most minimal WM and turning off even its title bars, it gets “one fullscreen window, no chrome, nothing to manage” — a kiosk — for free. A fun corner case: you can legally run X with *no* window manager at all, in which case windows appear wherever the app asks and can’t be moved.

5.3 Big fonts everywhere: .Xresources, Xft, and DPI

Children need large text, and X has two completely different font systems, both of which kidbox touches in [config/Xresources](#).

The modern path is **Xft** (the X FreeType interface), which renders scalable, anti-aliased TrueType fonts. kidbox raises the global DPI so everything scales up at once:

```
Xft.dpi: 120
Xft.antialias: true
Xft.hintstyle: hintfull
XTerm*faceName: Monospace
XTerm*faceSize: 24
```

Setting `Xft.dpi` to 120 (from the usual 96) is a global “make everything 25% bigger” lever for every Xft-aware app. The older path is the original X core fonts, addressed by those intimidating strings:

```
XTerm*font: -misc-fixed-medium-r-normal--20-200-75-75-c-100-iso10646-1
```

Going deeper: decoding an XLFD, field by field

That hyphen-salad is an **XLFD** — an X Logical Font Description — and it is not random. It has fourteen fields, each delimited by a hyphen, describing one specific bitmap font. Reading the example:

Field	Value (meaning)
foundry	misc (who made it)
family	fixed (the typeface)
weight	medium (vs. bold)
slant	r (roman; i=italic, o=oblique)
set width	normal
pixel size	20 (glyph height in pixels)
point size	200 (deci-points, i.e. 20.0 pt)
resolution x/y	75/75 (DPI the font was designed for)
spacing	c (character cell; m=monospace, p=proportional)
average width	100 (deci-pixels)
charset	iso10646-1 (Unicode)

The wildcard `*` can stand in for any field, which is how people usually write these (`--fixed-----20-----*`). kidbox spells out the full description to pin down *exactly* the 20-pixel fixed font it wants. The fact that a project for two children contains a hand-written XLFD is a small delight; almost nobody learns these any more.

5.4 The one escape hatch: Ctrl+Alt+Backspace via xbindkeys

There is exactly one intentional way to abandon a running activity, defined in [config/xbindkeysrc](#):

```
"pkill -TERM Xorg"
control+alt + BackSpace
```

Ctrl+Alt+Backspace runs `pkill -TERM Xorg`, which sends SIGTERM to the X server. When the server dies, `xinit` returns, `run_x` returns, and the menu loop redraws — so the child is safely back at the menu.

Going deeper: “zapping” X, and why this keystroke is special

Ctrl+Alt+Backspace is not an arbitrary choice. For most of X11’s history it was the built-in “zap” shortcut that instantly killed the X server — a vital panic button when a full-screen app wedged your machine. Around 2008, distributions disabled it by default (too many people hit it by accident and lost their work), moving it behind a keyboard option called `terminate:ctrl_alt_bksp` or the server’s `DontZapfrom` setting. kidbox can’t rely on that built-in being on, so it *re-implements the classic zap by hand* with `xbindkeys` and `pkill`. It’s a lovely historical echo: the project resurrects, in user space, a shortcut that was once baked into the X server itself — and uses it for exactly its original purpose, a guaranteed way out.

6 The activities

With the plumbing understood, the activities themselves are almost a victory lap. There are eight menu items; they fall into three groups.

6.1 Two ordinary desktop programs

Leafpad (menu items 1–2) is a tiny GTK text editor, opened on a fixed file per child — `typing/sally.txt` or `typing/penny.txt`. The installer is careful to create those files only if they don’t already exist, so a child’s typing survives reinstalls (more on that in the install section).

Tux Paint (item 3) is a children’s painting program from the early 2000s, complete with rubber-stamp clip art and sound effects. It needs no configuration; kidbox just runs it fullscreen.

6.2 Three vintage programming environments

These are the soul of the project — the “start understanding how it works” part made literal.

UCBLogo (item 4) is the turtle-graphics language. The menu opens it on `logo/welcome.lg`, which defines a couple of procedures and draws a house to greet the child.

Going deeper: Logo, the turtle, and “constructionism”

Logo was created in 1967 at BBN by Wally Feurzeig, Cynthia Solomon, and Seymour Papert. Papert — who had worked with Jean Piaget — believed children learn deepest when *building* things they care about, a philosophy he called *constructionism*. The “turtle” (originally a real floor robot dragging a pen, later a screen cursor) lets a child *become* the turtle: “how would I walk to draw a square?” becomes `REPEAT 4 [FD 100 RT 90]`. Underneath the playful syntax, Logo is a genuine dialect of **Lisp** — lists are first-class, procedures are values — so a child doodling with the turtle is quietly using one of computing’s most profound language families. UCBLogo, the implementation kidbox installs, was written by Brian Harvey at UC Berkeley to accompany his three-volume *Computer Science Logo Style*.

PC-BASIC (item 5) is a faithful emulator of **GW-BASIC**, the interpreter Microsoft shipped with early PCs. The starter program, `basic/HELLO.BAS`, is pure 1980s: numbered lines, `CLS`, `PRINT`, and an infinite `90 GOTO 90` loop you stop with Ctrl+Break.

```

10 CLS
20 PRINT "HELLO!"
80 PRINT "Press Ctrl+Break to stop."
90 GOTO 90

```

Going deeper: line numbers, GOTO, and a 60-year-old idea

BASIC — the Beginner’s All-purpose Symbolic Instruction Code — was created in 1964 at Dartmouth by John Kemeny and Thomas Kurtz, with a radical goal: that *every* student, not just engineers, should be able to write programs. The numbered lines kids type here (10, 20, 30...) are a relic of the era before screen editors: the number was both a label and a sort key, so you could insert 15 between 10 and 20 later. GOTO, later condemned by Dijkstra’s famous 1968 letter “Go To Statement Considered Harmful,” is here a feature, not a sin — the simplest possible way to show a child that a program can *loop*. PC-BASIC itself is a modern surprise: it is written in Python, yet emulates the original interpreter so closely that decades old .BAS files run unchanged. A child on this Pi is running, in effect, a 1983 Microsoft product reincarnated in a 2020s language.

6.3 Three home-grown activities

The clock (item 6) is a single self-contained HTML file (`content/clock.html`) shown through Chromium in kiosk mode. It draws an analog face on an HTML `<canvas>` and a digital readout beside it, updating once a second:

```

<canvas id="analog" width="700" height="700"></canvas>
...
setInterval(drawClock, 1000);

```

The hand angles are pleasant little trigonometry: the hour hand is at $(\text{hours}\%12 + \text{minutes}/60) * \pi/6$ radians, so it advances smoothly through the hour rather than jumping. `chromium-browser -kiosk -app="file://..."` is the same kiosk trick as the X session, one level up: a browser with no tabs, no address bar, no chrome — just the page.

Field note

The clock redraws on a `setInterval(..., 1000)` timer. A common refinement is `requestAnimationFrame`, which syncs redraws to the display and pauses when the page is hidden — but for a permanently-visible full-screen clock the simple interval is genuinely the right, “explicit” choice. Worth knowing the alternative exists; no need to change it.

The timer (item 7) asks for a number of minutes with a zenity dialog (a GTK pop-up, the graphical sibling of whiptail), validates it with a regular expression, then runs a countdown in a giant-font xterm. The countdown script (`bin/timer-countdown.sh`) loops once a second, optionally looping `timer.mp3` as background music through mpg123, and when it reaches zero clears the screen, prints `TIME’S UP!`, and plays `alarm.mp3`.

Field note

Two real rough edges in `timer-countdown.sh`, in the spirit of your “AI slop” hunt:

1. The alarm block guards on the *wrong* variable: `if [ -f "$TIMER_SOUND" ] && ...` then plays `$ALARM_SOUND`. So the alarm only fires if the *background-music* file exists, and if you ship an `alarm.mp3` but no `timer.mp3`, the alarm silently never plays. That guard almost certainly means to test `$ALARM_SOUND`.

2. The earlier `trap "kill $MUSIC_PID"` is overwritten by the later `trap "kill $ALARM_PID"`; bash traps don't stack, they replace. In practice the music is already killed explicitly before the alarm starts, so nothing leaks — but it's a footgun if the order ever changes.

Neither breaks the happy path (both MP3s shipped), which is why they survived.

The stopwatch (item 8) is the timer's mirror image: an `xterm` running a `while true` loop that counts up, formatting seconds into `MM:SS` (or `HH:MM:SS` past an hour) with `printf`.

The book (item 9) opens the instruction booklet PDF in the same Chromium kiosk mode. The README records a nice bit of real-world decision-making: `evince` took ~60 seconds to launch on the Pi, and `xpdf`'s interface was too fiddly for children, so a kiosk browser won.

Going deeper: zenity and the GTK dialog family

`zenity` is to graphical scripts what `whiptail` is to terminal scripts: a command that throws up a single dialog — entry box, error, file-chooser, progress bar — and reports the result on standard output, so a shell script can ask the user a question without becoming a “real” GUI program. It's built on GTK, the same toolkit behind Leafpad and much of the GNOME world. `kidbox` uses it for exactly one question (“how many minutes?”), which is `zenity`'s sweet spot. Its KDE-flavoured counterpart is `kdiallog`; both descend conceptually from the same “let scripts pop a dialog” idea as `dialog/whiptail`, just in pixels instead of text cells.

6.4 Sound: ALSA, amixer, and mpg123

The menu sets the volume to full at startup with two `amixer` calls (one for `Master`, one for `PCM`), each guarded with `|| true` so a missing control on some Pi audio configuration doesn't abort the script.

Going deeper: what ALSA is, and why two volume controls

ALSA — the Advanced Linux Sound Architecture — is the kernel's audio layer: the drivers and the `/dev/snd/*` devices that actually talk to the sound hardware. `amixer` is its command-line mixer, and `aplay` its player. The reason `kidbox` sets *both* `Master` and `PCM` is that a sound card exposes a *tree* of volume controls: `PCM` is typically the digital playback stream's level, while `Master` is the overall output; either one left low will keep things quiet, so the script maxes both. `mpg123` (used for the timer sounds) is a minimal command-line MP3 player — no GUI, just “decode this file to the sound card,” with `-loop -1` meaning “repeat forever.” Above ALSA, most desktops run **PulseAudio** or the newer **PipeWire** to mix multiple apps and route Bluetooth headsets; `kidbox` needs none of that and talks to ALSA directly — one fewer moving part.

7 The power button: a small system unto itself

The power button gets its own subsystem, and it's the most sophisticated piece of the project, so it earns the longest section. The goal is humane: a quick, accidental press should not kill the machine (and lose a child's drawing); it should ask first. Only a deliberate long hold should force power-off.

7.1 The problem and the three-part solution

By default, `systemd-logind` — the `systemd` component that manages login sessions and “special” keys — powers the machine off the instant the power button is pressed. `kidbox` overrides this in three coordinated pieces:

1. Tell **logind** to ignore the button (a config drop-in).
2. Watch the button ourselves and time the press (a Python service).
3. Show a confirmation menu on a short press (a shell script on a dedicated VT).

Figure 2 traces a press through all three.

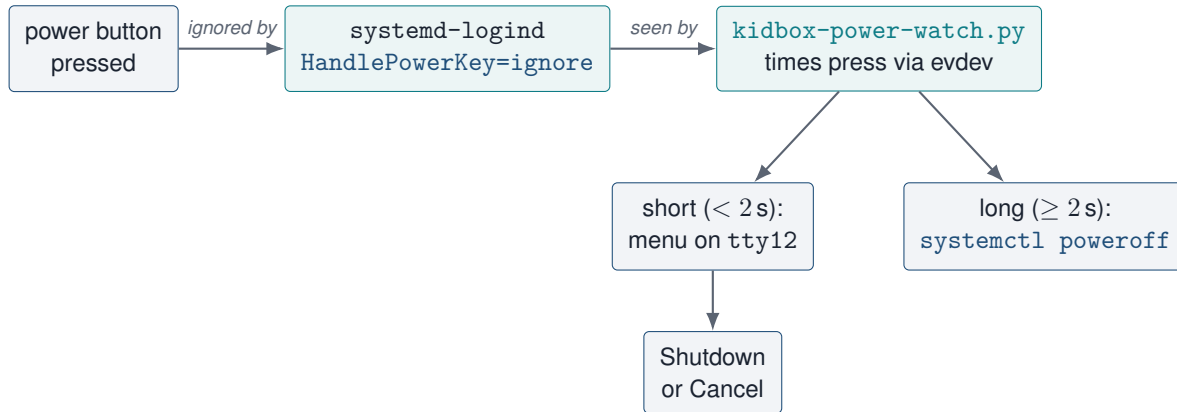


Figure 2: One button press, routed by three cooperating components.

7.2 Part 1: telling logind to stand down

The file `config/systemd/kidbox-power.conf` is installed as a *drop-in* at `/etc/systemd/logind.conf.d/kidbox-power.conf`:

```
[Login]
HandlePowerKey=ignore
# HandlePowerKeyLongPress=poweroff # systemd >= 245
```

`HandlePowerKey=ignore` tells logind to do nothing on the power key, ceding control to kidbox’s own watcher. The commented `HandlePowerKeyLongPress` line is a thoughtful note: newer systemd can itself distinguish a long press, but kidbox can’t assume that version, so it handles *both* press lengths in its own watcher instead.

Going deeper: drop-ins, and why you never edit the original file

Notice kidbox never edits `/etc/systemd/logind.conf` directly. Instead it drops a small file into a `.conf.d/` directory beside it. This is the pervasive systemd *drop-in* pattern: the main config plus every snippet in the matching `.d/` directory are merged at load time, with later files overriding earlier keys. The payoff is that package upgrades can replace the original file freely without clobbering your changes, and your change is a single self-describing file you can list, version, or delete. The same mechanism is what `systemctl edit getty@tty1` used earlier for autologin — it just writes an `override.conf` drop-in for you.

7.3 Part 2: watching the button with evdev

The watcher (`bin/kidbox-power-watch.py`) is the clever heart. It reads the kernel’s raw input events directly through **evdev**. First it finds every input device that claims to emit a power key:

```
for device_path in list_devices():
    device = InputDevice(device_path)
    caps = device.capabilities()
```



```
if ecodes.EV_KEY in caps and ecodes.KEY_POWER in caps[ecodes.EV_KEY]:
    devices.append(device)
```

Then it watches *all* of them at once with `select.select()` — blocking efficiently until any device has an event — and times the gap between the key-press (`value == 1`) and key-release (`value == 0`) events. A gap of ≥ 2 seconds is a long press (immediate poweroff); a shorter one launches the menu. A half-second debounce guards against electrical bounce.

The repository even keeps the throwaway probe script (`power.py`) that was clearly used to *discover* which devices report the power key:

```
for d in list_devices():
    dev = InputDevice(d)
    has_power = ecodes.KEY_POWER in dev.capabilities().get(ecodes.EV_KEY, [])
    print(f'{d}: {dev.name} (KEY_POWER={has_power})')
```

Going deeper: the Linux input subsystem and evdev

Every key, button, knob, and touchpad on a Linux machine surfaces through the **input subsystem** as a device file under `/dev/input/event*`. Reading one yields a stream of fixed-size *input events*, each a (type, code, value) triple: type `EV_KEY` for buttons, codes like `KEY_POWER` (numeric 116) for which key, value `1/0` for press/release (and `2` for autorepeat). `evdev` (“event device”) is this generic interface, and the Python `python3-evdev` package is a thin, pleasant wrapper over it. The reason the watcher monitors *every* device advertising `KEY_POWER` — rather than the first one — is a hard-won subtlety the code comments record: on real hardware several virtual devices (an ACPI button, a GPIO button, a PMIC) all *claim* the capability, but only one actually *fires*. Watching them all and reacting to whichever speaks is the robust answer, and the git history shows this was learned the hard way (“Monitor all `KEY_POWER` devices instead of just the first”).

Going deeper: the road not taken — `acpid` and `udev`

Before `systemd-logind`, the canonical way to handle a power button was **`acpid`**, the ACPI event daemon: it listened for ACPI events (lid, power, sleep) and ran shell scripts from `/etc/acpi/events/`. Many embedded projects still use it. A second alternative is a **`udev`** rule that runs a program when a specific input device appears. `kidbox`’s choice — a long-running service reading `evdev` itself — is heavier than a `udev` one-liner but gives it exactly what it needs: precise press *timing*, which neither `acpid`’s event-callback model nor a `udev` rule expresses naturally. It’s a good example of picking the tool whose *shape* matches the problem (“measure a duration”) rather than the most common one.

7.4 Part 3: the confirmation menu on its own VT

On a short press the watcher runs `bin/kidbox-power-menu.sh`, which does something delightfully sneaky. The watcher runs as a `systemd` service with *no controlling terminal*, but `whiptail` needs a terminal to draw on. So the script:

```
export TERM=linux # services inherit no TERM
ORIG_VT=$(cat /sys/class/tty/tty0/active | grep -o '[0-9]*$')
chvt 12 # jump to an unused VT
exec </dev/tty12 >/dev/tty12 2>&1 # attach all fds to it
whiptail --title "Power Button" --nocancel --menu ... 1 Shutdown 2 Cancel ...
```

It records the current VT (so it can return there on Cancel), switches the display to `tty12` with `chvt`, then

re-attaches its own standard input, output, and error to `/dev/tty12` so whiptail has a real terminal to read and draw on. Shutdown runs `systemctl poweroff`; Cancel (or Esc) `chvt`s back to wherever the child was. Using a dedicated, otherwise-empty VT means the power dialog and the main menu (on `tty1`) can never collide for the keyboard — a genuinely elegant solution to “two programs, one keyboard.”

7.5 The service unit, and what its hardening buys

The watcher is kept alive by `kidbox-power-watch.service`:

```
[Service]
Type=simple
ExecStart=/usr/local/bin/kidbox-power-watch.py
Restart=always
RestartSec=5
ProtectSystem=strict
ProtectHome=yes
PrivateTmp=yes
DevicePolicy=auto
```

`Restart=always` means `systemd` revives the watcher if it ever dies — essential for something guarding the power button. The rest is *sandboxing*, and it’s worth knowing what each line actually does, because these directives are `systemd`’s quiet superpower:

- `ProtectSystem=strict` — mounts the *entire* filesystem read-only for this process, except `/dev`, `/proc`, `/sys`. The service literally cannot modify a file anywhere unless explicitly allowed.
- `ProtectHome=yes` — makes `/home` (and `/root`, `/run/user`) appear *empty* to the process. A power watcher has no business reading anyone’s files, so it can’t.
- `PrivateTmp=yes` — gives the service a private `/tmp` that vanishes when it stops, so it can’t be used to leak or stash data.
- `DevicePolicy=auto` — the device-access policy that still lets it reach the input devices it genuinely needs.

Going deeper: services as lightweight sandboxes

These directives lean on the same kernel primitives — mount namespaces, cgroups, capability dropping — that containers like Docker are built from. `systemd` quietly exposes them one keyword at a time, so an ordinary service can be locked down to “can read input devices, can’t touch the disk or anyone’s home directory” without any container runtime at all. For a process that runs as `root` (which this one must, to read raw input devices), that defence in depth is exactly right: if the watcher were ever compromised, the blast radius is deliberately tiny. You can audit any unit’s effective sandboxing with `systemd-analyze security kidbox-power-watch.service`, which scores each setting — a genuinely fun command to run on your own machine.

Field note

A small drift between docs and code: the README and the unit’s own header comment still describe the menu as “Shutdown / Reboot / Cancel,” but the git history (“Remove reboot option from power menu”) and the current script offer only *Shutdown* and *Cancel*. The behaviour is the intended one; the comments just haven’t caught up. A one-line edit to the README and the `.service` header would close the gap.

8 Permissions: a deliberately small amount of power

kidbox grants the locked-down girls account exactly one privileged ability, and does it carefully. The installer writes a sudoers drop-in:

```
SUDOERS_FILE="/etc/sudoers.d/kidbox-shutdown"
cat > "$SUDOERS_FILE" <<EOF
girls ALL=(ALL) NOPASSWD: /sbin/shutdown -h now
EOF
visudo -c -f "$SUDOERS_FILE" || { rm -f "$SUDOERS_FILE"; exit 1; }
chmod 0440 "$SUDOERS_FILE"
```

This lets girls run *only* the exact command `/sbin/shutdown -h now`, and only that — not `shutdown` with other arguments, not anything else — and without a password (the account has none). The menu’s “Shutdown” item uses precisely that command.

Three things here are textbook-correct and worth internalising:

- **A drop-in, not an edit to `/etc/sudoers`.** Same reasoning as the systemd drop-ins: upgrades and tooling stay clean, and the grant is one self-contained, removable file.
- **`visudo -c` validates before activating.** A syntax error in a live sudoers file can lock *everyone* out of sudo. The installer validates the file and *deletes it on failure* rather than leave a broken rule in place. This is the single most important habit in sudoers editing, and the script gets it right.
- **Mode 0440.** sudo refuses to read a sudoers file that is group- or world-writable (a security check); 0440 (read-only for owner and group, nothing for others) is the required permission.

Going deeper: the threat model is “a toddler,” not “an attacker”

It’s worth being honest about what this security is *for*. The passwordless account, the inescapable menu, the trapped Ctrl+C — none of it would stop a determined teenager with a keyboard and the internet. That’s fine, because the threat model is a five-year-old who might wander into a shell, delete something, or shut the machine off mid-drawing. kidbox is built to prevent *accidents and confusion*, not *adversaries*. Recognising which kind of security you’re building is half of doing it well; over-engineering a child’s computer against nation-state attackers would be its own kind of mistake. The group memberships the installer grants (audio, video, input, plugdev, render) follow the same restraint: exactly the hardware-access groups a desktop user needs, and nothing like sudo or adm.

9 Installation: one idempotent shell script

`install.sh` (originally drafted, its header notes, with ChatGPT’s help) turns a stock Raspberry Pi OS into a kidbox. It’s a model of how to write an installer in plain bash, and several of its techniques are reusable far beyond this project.

9.1 Safety rails at the top

```
#!/usr/bin/env bash
set -euo pipefail
[[ $EUID -ne 0 ]] && { echo "Please run as root."; exit 1; }
KID_USER="${KID_USER:-girls}"
```

`set -euo pipefail` is the “strict mode” every serious shell script should open with: `-e` exit on any unhandled error, `-u` treat unset variables as errors (catching typos), `pipefail` make a pipeline fail if *any* stage fails, not just the last. The root check fails fast and friendly. `KID_USER:-girls` reads an environment

variable with a default, which is why `sudo KID_USER=girls ./install.sh` works.

9.2 Idempotency: safe to run again and again

The README tells you to *update* by re-running the installer, so every step must be safe to repeat. The script achieves this in several ways:

- It creates the user only if `! id -u "$KID_USER"` — skipping if present.
- It uses the `install(1)` command (not `cp`) to copy files with explicit permissions in one atomic step — e.g. `install -m 0755` for scripts, `install -m 0644` for data, `install -d` to create directories.
- For the children’s *typing* files it copies *only if the destination is missing*, so a child’s saved text is never clobbered by an update — while BASIC and Logo example files *are* overwritten, because they’re templates.

9.3 The marker-block trick for `.bash_profile`

The most quietly clever part is how the installer edits `.bash_profile` without ever duplicating its block on re-runs. It wraps its snippet in marker comments and, before re-adding, strips any previous block with `awk`:

```
MARK_BEGIN="# >>> kidbox menu autostart >>>"
MARK_END="# <<< kidbox menu autostart <<<"
awk -v b="$MARK_BEGIN" -v e="$MARK_END" '
    $0==b {inblock=1; next}
    $0==e {inblock=0; next}
    !inblock {print}
    ' "$BASH_PROFILE" > "$tmp"
# ... then append a fresh marked block
```

The `awk` program prints every line *except* those between the markers, deleting any prior install. Then a fresh block is appended. Run the installer ten times and you still get exactly one block. If those “>>>/<<<” markers look familiar, it’s because this is the very same pattern `conda`, `nvm`, and countless other tools use to manage their lines in your shell config — a small, robust convention worth keeping in your own toolbox.

9.4 Then it wires up the system

The tail of the script installs the `systemd` units, the `sudoers` file, and the `logind` drop-in we’ve already met, then `systemctl daemon-reload`, enables and starts the power watcher, and restarts `systemd-logind` to pick up the new config. The package list at the top (`apt-get install` of `ucblogo`, `python3-pcbasic`, `tuxpaint`, `xserver-xorg`, `xinit`, `matchbox-window-manager`, `whiptail`, `xbindkeys`, `chromium-browser`, `mpg123`, `python3-evdev`, and `friends`) is the complete bill of materials for everything described in this report.

Going deeper: the road not taken — configuration management

A shell installer is the simplest possible answer to “put these files here and flip these switches.” The industrial alternative is a *configuration management* tool — Ansible, Puppet, Salt — where you describe the desired *end state* (“this file exists with this content and mode; this service is enabled”) and the tool figures out how to get there, idempotently, by design. Those tools shine when you manage a hundred machines and need declarative guarantees. For one Pi in a child’s room, a 240-line bash script you can read top to bottom is not just adequate, it’s *more* in keeping with the project’s “simple and explicit” creed than a YAML playbook would be. Still, it’s instructive to notice that the script is hand-rolling, in imperative shell, exactly the idempotency (create-if-missing, copy-if-changed, edit-once) that Ansible would give you declaratively for free — a nice lens on what those big tools are actually *for*.

10 The instruction booklet: LaTeX and a little bookbinding

The last subsystem is the one you’ll physically hold: `doc/kidbook.tex`, a LaTeX document typeset and then *imposed* into a foldable, staplable booklet. It’s a charming corner of the project because it reaches all the way from digital typesetting into paper craft.

10.1 A book sized for small hands

The document class is KOMA-Script’s `scrbook`, with a custom page size:

```
\documentclass[11pt,oneside,paper=5.5in:8.5in,DIV=13,
chapterprefix=false,headings=big]{scrbook}
```

`paper=5.5in:8.5in` is *half* of US Letter — the natural page size for a booklet folded from letter paper. `DIV=13` is KOMA-Script’s *type-area* control: rather than set margins directly, you tell it into how many strips to divide the page, and it computes classically-proportioned margins for you. `oneside` (versus `twoside`) keeps margins symmetric, which matters once the pages are imposed.

10.2 From PDF to bootable booklet: imposition

The `doc/Makefile` builds two PDFs:

```
pdf: $(TEX).tex
    pdflatex $(TEX).tex
    pdflatex $(TEX).tex          # twice, to resolve the table of contents
booklet: pdf
    pdfjam --booklet true --landscape --paper letterpaper $(PDF) -o $(BOOKLET)
```

The first target produces an ordinary, page-1-2-3-4 PDF at half-letter size. The second runs `pdfjam` (a wrapper around the `pdfpages` LaTeX package) with `-booklet`, which performs **imposition**: it reorders and 2-up’s the pages onto landscape letter sheets so that, when you print double-sided, fold the stack in half, and staple the spine, the pages come out in reading order. The README’s printing instructions (duplex, long-edge binding, fold horizontally, staple the fold) are the physical other half of that algorithm.

Going deeper: signatures, imposition, and why page 1 sits next to the last page

Imposition is older than computers — it’s how every paperback you own was made. Fold one sheet of paper in half and you get four pages (two per side); the trick is that those four pages are *not* 1, 2, 3, 4 in sequence on the flat sheet. On the outside of a single folded sheet sit page 1 and the *last* page, back to back with page 2 and the second-to-last. A folded bundle like this is called a **signature**, and real books are sewn from many signatures in a row. This is exactly why the booklet must have a page count that’s a multiple of four (you can even see a commented-out “repeat until page count is a multiple of 4” reminder at the end of `kidbook.tex`), and why `pdfjam` has to compute that surprising 1-next-to-last ordering. The same maths that lays out a medieval manuscript lays out this Raspberry Pi instruction book.

Going deeper: what TeX is, and why we run it twice

$\text{\TeX}$, Donald Knuth’s typesetting system from the late 1970s, was born of his disgust at the typography of his own books’ second edition; he decided to fix typesetting itself and didn’t publish the next volume for a decade. $\text{\LaTeX}$ (Leslie Lamport, 1980s) is a layer of sensible document-structure macros on top. The reason the `Makefile` runs `pdflatex` *twice* is a famous $\text{\TeX}$ quirk: the first pass discovers where every chapter and page lands and writes those numbers to an auxiliary file; the second pass reads

that file back to build a correct table of contents and cross-references. Complex documents sometimes need three passes, or the `latexmk` tool, which simply keeps running `TEX` until the auxiliary files stop changing. This very report you’re reading was built the same way.

11 The development story in the repository

A last, more human layer: the repository itself tells the story of how kidbox was built, and it matches your description of the work.

The `.claude/` directory holds a `README.md` that briefs an AI assistant on the project’s philosophy and structure, plus two *slash-command* definitions — `build-docs` (“run make in `doc/`”) and `check-structure` (“verify the key files exist”). These are little reusable instructions for the assistant, the modern equivalent of a Makefile target written in English.

The git history is candid and instructive. You can watch the power button feature being fought into existence over a run of commits — “Attempt to get the power button service to work,” “Monitor all `KEY_POWER` devices instead of just the first,” “Fix power menu: monitor all devices, use dedicated VT12” — which is precisely the `evdev/multi-device` lesson documented earlier. There are honest commits named `foobar` and `tmp script`, a “Fix AI slop; better way of setting size,” and a tidy set of merge commits from `claude/`-prefixed feature branches. It is a faithful record of vibe-coding with a careful human steering: each AI-generated lurch followed by a correcting hand.

Going deeper: “simple and explicit” as an engineering discipline

Step back and the whole project rhymes. Every layer chooses the small, legible, inspectable option over the powerful, opaque one: a shell installer over Ansible; `xinit` over a display manager; `matchbox` over a desktop environment; `whiptail` over a graphical kiosk framework; an `evdev` script over a black-box appliance; raw `sudoers` and `systemd` drop-ins over some management layer. Each individual choice is defensible on its own (lighter on the Pi, fewer dependencies), but together they express a single coherent value: a machine whose every behaviour can be traced to a file a curious person — child or parent — can open and read. That is the same promise the README makes to its users, kept by the engineering all the way down. It is, quietly, a rather elegant piece of work.

Appendix A: File map

Appendix B: A field guide to the commands and concepts

`agetty`

The program a `getty@` service runs to offer a console login; `-autologin` skips authentication.

ALSA / `amixer`

The kernel sound layer and its command-line mixer.

`chvt`

Switch the active virtual terminal from the command line.

DPMS

Display Power Management Signaling — how X tells a monitor to sleep.

`drop-in`

A small config file in a `.d/` directory, merged over a main config; used here for `logind`, `getty`, and `sudoers`.

`evdev`

The Linux generic input-event interface under `/dev/input/event*`.

Path	Role
install.sh	Idempotent system installer (root).
power.py	One-off probe: which devices report KEY_POWER.
bin/menu.sh	The whiptail main menu loop.
bin/clock.sh	Launches the HTML clock in Chromium kiosk.
bin/timer.sh	Asks minutes (zenity), runs the countdown.
bin/timer-countdown.sh	The per-second countdown + sound.
bin/stopwatch.sh	Count-up timer in a big-font xterm.
bin/kidbox-power-watch.py	evdev power-button watcher (systemd service).
bin/kidbox-power-menu.sh	Power confirmation menu on tty12.
config/xinitrc	Minimal X session (WM, hotkeys, one app).
config/Xresources	Large fonts / DPI for X apps.
config/xbindkeysrc	Ctrl+Alt+Backspace → kill X (escape hatch).
config/bash_profile_snippet.sh	Console font + launch menu at login.
config/systemd/*	logind drop-in and the watcher service unit.
content/clock.html	Self-contained analog+digital clock.
content/basic/HELLO.BAS	Starter BASIC program.
content/logo/welcome.lg	Starter Logo procedures.
content/typing/*.txt	Per-child typing files (preserved on update).
doc/kidbook.tex	The instruction booklet source.
doc/Makefile	Builds the half-letter PDF and imposed booklet.

getty / tty

“Get tty”; a tty is a terminal, named for the teletypewriter.

idempotent

Safe to run repeatedly with the same result — the installer’s design goal.

imposition

Reordering pages so a folded, stapled stack reads correctly.

matchbox

A minimal window manager for small screens.

NEWT_COLORS

Environment variable that recolours whiptail; abused here to hide the OK button.

systemd-logind

The systemd service managing login sessions and power keys.

XLFD

X Logical Font Description — the 14-field hyphenated core-font name.

xinit

Starts an X server plus one client and exits when that client does.

xrdb

Loads the X resource database ([.Xresources](#)).

*End of report. Built with `pdflatex`; typeset in Bitstream Charter.
Sleep well, and may your children never find a shell prompt.*